

Lists in variables

There's something about how Python stores lists in variables that might seem a bit odd at first. But take a look at what's going on behind the scenes and it all makes sense.

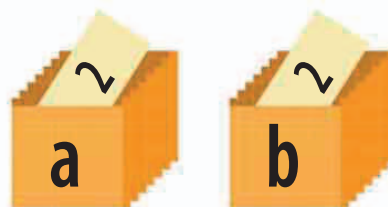
SEE ALSO

◀ 108–109 Variables in Python

◀ 128–129 Lists

Remember how variables only store values?

Variables are like boxes that hold values. The value in one variable can be copied and stored in another. It's like photocopying the value contained in box "a" and storing a copy in box "b".



△ How variables work

Each variable is like a box containing a piece of paper with a value written on it.

1 Assign a value to a variable

Assign the value 2 to variable "a", then assign the value in "a" to variable "b". The value 2 is copied and stored in "b".

```
>>> a = 2
>>> b = a
>>> print('a =', a, 'b =', b)
a = 2 b = 2
```

This copies the contents of "a" into "b"

Now "a" and "b" both contain the value 2

This prints out the variable names with their values

2 Change a value

If you change the value stored in one variable it won't affect the value stored in another variable. In the same way, changing what's written on a piece of paper in box "a" won't affect what's on the paper in box "b".

```
>>> a = 100
>>> print('a =', a, 'b =', b)
a = 100 b = 2
```

Change the value in "a" to 100

Now "a" contains 100, but "b" still contains 2

3 Change a different value

Change the value in "b" to 22. Variable "a" still contains 100. Even though the value of "b" was copied from "a" at the start, they are now independent—changing "b" doesn't change "a".

```
>>> b = 22
>>> print('a =', a, 'b =', b)
a = 100 b = 22
```

"b" now contains 22, but "a" is still 100